

Team Formation

You must form groups of 3–4 people. Please email harrisen.scells@uni-tuebingen.de with a list of team members by the 02.07.2026 *at the latest*. A group will be created for you in ILIAS to submit the project. You are allowed to create groups however you like. Please feel free to use the discussion board in ILIAS to help find group members.

Deliverables

- 4-page report
- Code repository
- User interface

The 4-page report should clearly contain a link to the code repository (to a branch that does not change after the submission deadline) and several screenshots of the user interface. Please submit the report to your *project group link* via ILIAS (one group submission per project team).

You can use any general-purpose ML and NLP libraries such as `scipy`, `numpy`, `scikit-learn`, `spacy`, `nlTK`, but please stay away from dedicated web crawling or search engine toolkits such as `scrapy`, `whoosh`, `lucene`, `terrier`, `galago` and the likes. Pretrained models are fine to use as part of your system, as long as they have not been built/trained for retrieval.

Grading

Your final projects will be graded along the following criteria:

- 25% Report (4 pages, PDF, explanation and justification of your design choices)
- 25% Code correctness and quality
- 25% System effectiveness (based on how well your system performs on five queries relative to the other teams in terms of nDCG)
- 15% Creativity and innovativeness of your approach; in particular with respect to your search system (Component 2) and user interface (Component 3).
- 10% Presentation quality and clarity

The following sections describe the four required components of the project. Use the provided code blocks to help you get started.

1 Web Crawling and Indexing

Crawl the web to discover *English content related to Tübingen*. The crawled content should be stored locally. If interrupted, your crawler should be able to re-start and pick up the crawling process at any time.

```
#Add a document to the index. You need (at least) two parameters:
```

```
    #doc: The document to be indexed.
```

```
    #index: The location of the local index storing the discovered documents.
```

```
def index(doc, index):
```

```
    #TODO: Implement me
```

```
    pass
```

```
#Crawl the web. You need (at least) two parameters:
```

```
    #frontier: The frontier of known URLs to crawl. You will initially populate this with your seed  
               set of URLs and later maintain all discovered (but not yet crawled) URLs here.
```

```
    #index: The location of the local index storing the discovered documents.
```

```
def crawl(frontier, index):
```

```
    #TODO: Implement me
```

```
    pass
```

2 Query Processing

Process a textual query and return the 100 most relevant documents from your index. Please incorporate *at least one retrieval model innovation* that goes beyond BM25 or TF-IDF. Please allow for queries to be entered either individually in an interactive user interface (see also Component 3 below), or via a batch file containing multiple queries at once. The batch file (see below for an example) will be formatted to have one query per line, listing the query number, and query text as tab-separated entries. An example of the batch file for the first two queries looks like this:

```
1  tübingen attractions
2  food and drinks
```

```
#Retrieve documents relevant to a query. You need (at least) two parameters:
    #query: The user's search query
    #index: The location of the local index storing the discovered documents.
def retrieve(query, index):
    #TODO: Implement me
    pass
```

3 Search Result Presentation

Once you have a result set, we want to return it to the searcher in two ways: a) in an interactive user interface. For this user interface, please think of *at least one innovation* that goes beyond the traditional 10-blue-links interface that most commercial search engines employ. b) as a text file used for batch performance evaluation. The text file should be formatted to produce one ranked result per line, listing the query number, rank position, document URL and relevance score as tab-separated entries. An example of the first few lines of such a text file looks like this:

```
1  1  https://www.tuebingen.de/en/3521.html  0.725
1  2  https://www.komoot.com/guide/355570/castles-in-tuebingen-district  0.671
1  3  https://www.unimuseum.uni-tuebingen.de/en/museum-at-hohentuebingen-castle  0.529
...
1  100 https://www.tuebingen.de/en/3536.html  0.178
2  1  https://www.tuebingen.de/en/3773.html  0.956
2  2  https://www.tuebingen.de/en/4456.html  0.797
...
```

```
#TODO: Implement an interactive user interface for part a of this exercise.
```

```
#Produce a text file with 100 results per query in the format specified above.
```

```
def batch(results):
    #TODO: Implement me.
    pass
```

4 Evaluation

We will evaluate the effectiveness of our search systems on the basis of five queries. Two of them are available to you now for engineering purposes (tab-separated file):

```
1  tübingen attractions
2  food and drinks
```

The remaining three queries will be given to you during our final session on 06.08.2026. Please be prepared to run your systems and produce a single result file for all five queries live in class. That means you should aim for processing times of no more than ~1 minute per query. We will ask you to send the file to harrisen.scells@uni-tuebingen.de.

Frequently Asked Questions

1. **What libraries are allowed for crawling?**
Libraries such as requests and beautifulsoup are permitted. Dedicated crawling frameworks such as Scrapy are not allowed.
2. **Can we index non-English pages?**
No. Only index English-language content. Non-English pages will be considered irrelevant.
3. **What counts as a retrieval model innovation?**
Your first-stage retrieval must be a classical approach you implement yourself (e.g., BM25). Any more sophisticated approach (e.g., neural re-rankers, embedding-based retrieval) should be used as a second-stage re-ranker on top of the classical first stage.
4. **Can we use FAISS or similar embedding libraries?**
Yes, but only as a second-stage component. You must still implement a classical first-stage retrieval method yourself.
5. **Can we use general-purpose pretrained models (e.g., BERT, Llama)?**
Yes, as long as the model has not been fine-tuned on a dedicated retrieval dataset (e.g., MS MARCO). General-purpose language models used for encoding or re-ranking are permitted; retrieval-specific models are not.
6. **Can we train a re-ranker ourselves?**
Yes. Fine-tuned pre-trained re-rankers are not permitted, but training a re-ranker from scratch is allowed.
7. **Can we implement something like image search?**
Yes. Models such as CLIP are permitted for this purpose, as long as they are not used as the sole retrieval system. A classical text-based retrieval stage is still required.
8. **Does the user interface have to be a web application?**
No. A command-line interface is acceptable, but it should be well-designed and easy to use. Remember that the innovation requirement still applies: your interface should offer something beyond a plain list of results.
9. **How will search results be evaluated?**
We will pool the top-n results submitted by all teams, manually assess them for relevance, and compute nDCG scores. Your system will be ranked relative to other teams.
10. **Do I have to use Python? Can I use my favourite programming language?**
You must use Python to implement everything in this project. Since part of your grade is based off code correctness and quality, we want to grade everyone fairly.